

Diseño del proyecto - Borrador de la documentación

Título del Proyecto: Generador de Diagramas de Clases UML (Vía Mermaid JS)

Integrantes: Deraz Labrador Emmanuel (2208862) y Salazar Leal Javier Issac (2208260)

La herramienta está diseñada para leer y procesar código en lenguaje Java. El sistema será capaz de identificar y extraer los siguientes elementos para la construcción del diagrama en mermaid:

- **Clases:** Identificación del nombre de cada clase del código.
- **Atributos:** Detección de las variables de instancia, sus tipos de datos y modificadores de acceso.
- **Métodos:** Identificación de funciones asociadas a la clase y sus parámetros.
- **Relaciones:** Detección de relaciones de herencia y relaciones de composición/agregación mediante el análisis de las tablas de símbolos y referencias de tipos.

Lógica a implementar

El software se estructurará sobre la base del analizador construido para la clase, adaptando sus fases lógicas para el procesamiento de Java:

- **Fase de Análisis Léxico:** El sistema leerá el archivo fuente de Java y separará el texto en palabras clave y símbolos. Para esto, se modificará la clase TipoToken y el PseudoLexer para reconocer los patrones de Java (ej. class, public, private, llaves, tipos de datos) en lugar de las palabras reservadas del pseudocódigo actual.
- **Fase de Análisis Sintáctico:** A través de una adaptación del PseudoParser, el programa revisará que los tokens sigan la estructura básica esperada para la declaración de clases, variables y métodos en Java.
- **Extracción y Tabla de Símbolos:** Se reutilizará y extenderá la infraestructura actual del paquete pseudo.symbols.
 - Las clases se registrarán utilizando ClassSymbol, aprovechando su capacidad para guardar la superclase (superClass) y rastrear la herencia.

- Los atributos se guardarán como instancias de **VariableSymbol**.
- Los métodos se gestionarán con **MethodSymbol**.

Reglas de Traducción

Elemento detectado en Java	Representación Interna en el Analizador	Sintaxis de Salida (Mermaid JS)
public class Nombre	ClassSymbol	class Nombre {
private int edad;	VariableSymbol dentro de la clase	-int edad
public String nombre;	VariableSymbol dentro de la clase	+String nombre
protected void hacer();	MethodSymbol dentro de la clase	#hacer() void
class A extends B	ClassSymbol con atributo superClass	B < -- A
Agregación / Composición	Tipo de VariableSymbol refiriendo a otro ClassSymbol	ClaseA *-- ClaseB

Entrada esperada (Código fuente Java a analizar):

```
Java
public class Estudiante extends Persona {
    private int matricula;
    public String nombre;

    public void estudiar() {
        // contenido del metodo
    }
}
```

Salida generada por el sistema (Texto Mermaid):

```
Java
classDiagram
    Persona <|-- Estudiante
    class Estudiante {
        -int matricula
        +String nombre
        +estudiar() void
    }
```

Generación del Archivo Final (Salida HTML)

El programa generará un archivo salida.html con la siguiente estructura base, donde inyectará dinámicamente el resultado:

```
Java
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Diagrama de Clases Generado</title>
</head>
<body>
    <h2>Diagrama UML del Proyecto</h2>
    <pre class="mermaid">
        classDiagram
            Persona <|-- Estudiante
            class Estudiante {
                -int matricula
                +String nombre
                +estudiar() void
            }
    </pre>
```

```

    }
  </pre>
  <script type="module">
    import mermaid from
    'https://cdn.jsdelivr.net/npm/mermaid@10/dist/mermaid.esm.min.mjs';
    mermaid.initialize({ startOnLoad: true });
  </script>
</body>
</html>

```

Demostración de cómo se vería la representación visual del código lado a lado, donde el código de mermaid ya está implementado en el HTML

index.html
Auto-ejecutar

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Diagrama de Clases Generado</title>
</head>
<body>
  <h2>Diagrama UML del Proyecto</h2>
  <pre class="mermaid">
    classDiagram
      Persona <|-- Estudiante
      class Estudiante {
        -int matricula
        +String nombre
        +estudiar() void
      }
  </pre>
  <script type="module">
    import mermaid from
    'https://cdn.jsdelivr.net/npm/mermaid@10/dist/mermaid.es
    m.min.mjs';
    mermaid.initialize({ startOnLoad: true });
  </script>
</body>
</html>

```

Vista Previa

Diagrama UML del Proyecto

```

classDiagram
    class Persona
    class Estudiante {
        -int matricula
        +String nombre
        +estudiar() void
    }
    Persona <|-- Estudiante

```